# Generative AI & Prompt Engineering Internship

**Skillify**




## Task 1

Nayab Maryam

Submitted to

Mam Momina Arshad

# Task 1: AI Career Assitent:

Objective

- Develop a simple using Python and Ollama.
- Generate relevant answers to career-related questions.
- Define the AI's role and behavior using a .
- Understand basic interaction between Python and an AI model.

Requirements

- Python
- Ollama
- Ollama AI Model
- career_assistant.py
- System Prompt
- User Input Handling

Submission Format

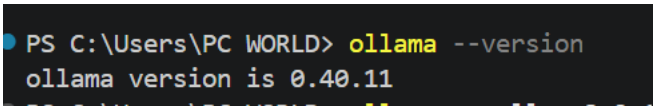
- Create a pdf containing
- career_assistant.py
- 2–3 screenshots of the working program.
- At least 5 career-related questions with their AI responses.

# Solution:

First I installed the Ollama model using this link:

Ollama · Run AI models locally and in the cloud

Then,I ran the command for finding Python and Ollama version:

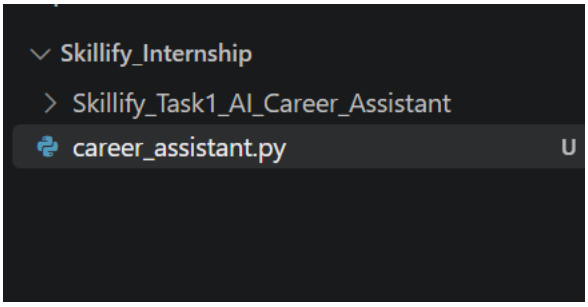



Then I run the below command to download AI model:

ollama run llama3.2:1b --command

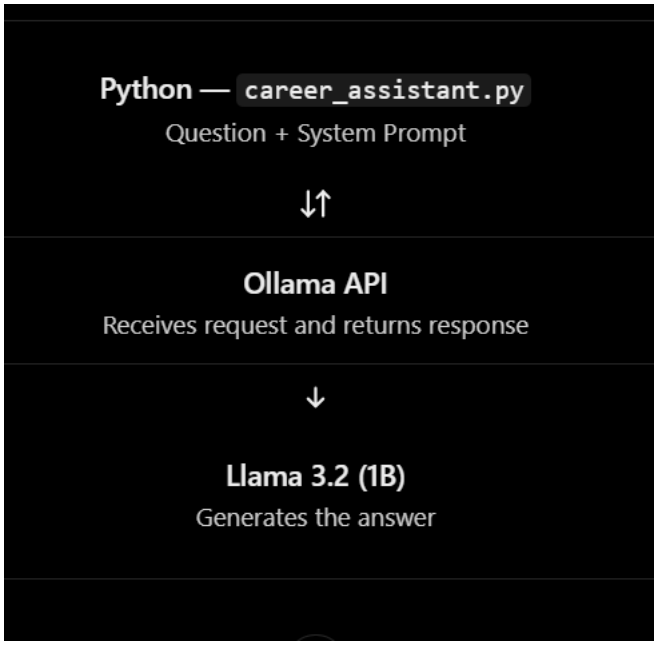



Then,I build a file named career_assistant.py



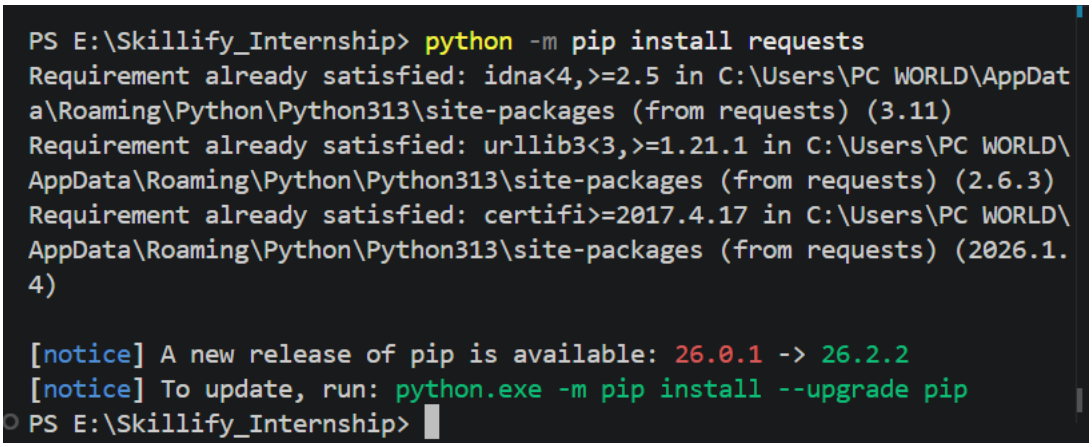

Now,here the design login I applied is:

User will ask the career related question ,python file is the main controller will receive it and send it to Ollama AI Model for generating response and then python file will receive that response and display it on the screen to the user.




- Python: Hamara program jo hum bana rahe hain.
- API: Ek communication interface jo do software components ko aapas mein baat karne deta hai.
- Ollama: Hamari request ko local AI model tak pohanchata hai.
- Llama: User ke question ka jawab generate karta hai.

Hum Python ki built-in urllib library ya requests library se API call kar sakte hain. Beginner-friendly aur simple implementation ke liye main filhaal requests recommend karungi.




**requests** Python ki library hai jo HTTP requests send karne mein madad karti hai. Hum isay Ollama API ko question bhejne ke liye use karenge.

Now I had written the code of career_assiatant.py:

First, I made an object of SYSTEM_PROMPT in which I write the rules of answering the query like user friendly ,briefly etc

Then I import request library to send HTTP requests then I build a function named get_career_response(user_question) this function will receive the user's question and then send it with the system prompt to the ollama model and it returns the response in one whole chunk instead of small chunks and if any HTTP error occurs it handles it by showing it

Then ,I coded a main where I added some greeting,welcome messages with how to exit the chatbot or how to ask question rules where u can enter exit to exit from the code also added some goodbye msg to make it more interative and most importantly I used the loop so that only when the user enters "exit" the chatbot terminates.

Following are the code and some career question I asked to my AI career _Assistant.py chatbot:

```
# SYSTEM_PROMPT an object use to write the instruction sof AI Assitant and """ is used for multline string
# System Prompt for the AI Career Assistant
SYSTEM_PROMPT = """
You are a helpful AI Career Assistant.
You provide practical and beginner-friendly career guidance.
You explain career paths, required skills, learning roadmaps,
and interview preparation.
Your answers should be clear, professional, and easy to understand.
If you are unsure about something, be honest instead of guessing.
"""


import requests  # To send HTTP requests we need this library

def get_career_response(user_question): # This function will recieve the users's question
    response = requests.post( # this will send the HTTP Post to Ollama API
        "http://localhost:11434/api/chat",  # This is the address of Ollama Chat API which recieves the question ad returns chat reponse
        json={
            "model": "llama3.2:1b",
            "messages": [
                {"role": "system", "content": SYSTEM_PROMPT}, # System prompt (rules) are send using role of system
                {"role": "user", "content": user_question}    # Users's question message send with role user
            ],
            "stream": False  #the response will be whole instaed of in chunks
        }
    )

    response.raise_for_status()  #  this will raises the error if any   HTTP error occurs
    return response.json()["message"]["content"]


def main():
    print("Welcome to AI Career Assistant!")
    print("Ask me anything about careers.")
    print("Type 'exit' to quit.")
```

```
    while True:
        user_question = input("\nYou: ").strip()

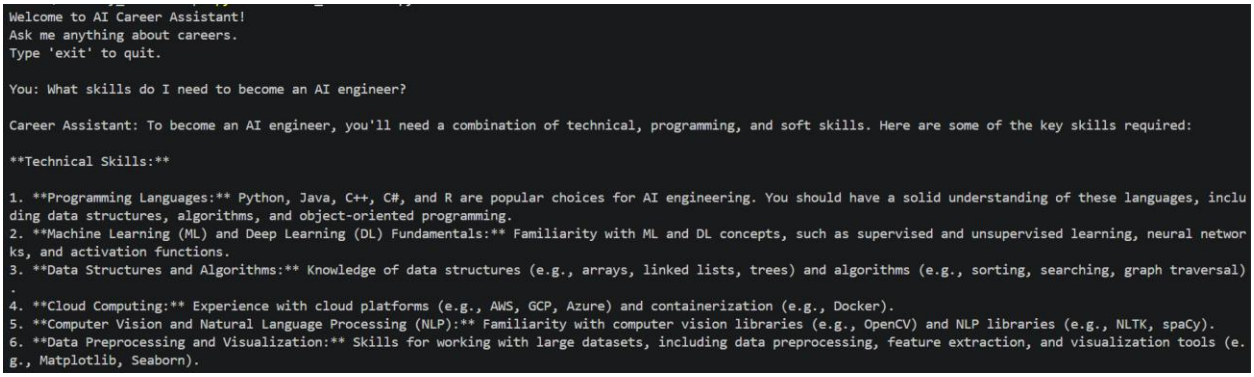
        if user_question.lower() == "exit":
            print("Career Assistant: Goodbye!")
            break

        if not user_question:
            print("Career Assistant: Please enter a question.")
            continue

        answer = get_career_response(user_question)
        print("\nCareer Assistant:", answer)


if __name__ == "__main__":
    main()
```

Question 1: What skills do I need to become an AI engineer?




Question 2: What skills do I need to become a backend developer?

```
You: What skills do I need to become a backend developer?

Career Assistant: To become a backend developer, you'll need to possess a combination of technical skills, knowledge, and soft skills. Here are some of the key s
ills required:

**Technical Skills:**

1. **Programming languages:** Proficiency in at least one programming language such as:
        * Python
        * Java
        * JavaScript
        * C#
        * Ruby
        * PHP
2. **Backend frameworks:** Familiarity with popular backend frameworks like:
        * Express.js (Node.js)
        * Django (Python)
        * Ruby on Rails
        * Laravel (PHP)
        * Spring Boot (Java)
        * ASP.NET Core (C#)
3. **Databases:** Understanding of various databases, including:
        * Relational databases (e.g., MySQL, PostgreSQL)
        * NoSQL databases (e.g., MongoDB, Cassandra)
4. **Server-side technologies:** Knowledge of server-side technologies like:
```

Question 3: What are the most promising career in artificial Intelligence?

```
You: What are the most promising careers in artificial intelligence?

Career Assistant: There are many promising careers in artificial intelligence (AI), and the field is constantly evolving. Here are some of the most promising care
ers in AI:

1. **Machine Learning Engineer**: Designs and develops models that enable machines to learn from data, making them more accurate and efficient.
2. **Data Scientist**: Uses AI and machine learning to analyze and interpret complex data, often working with large datasets to identify patterns and trends.
3. **Natural Language Processing (NLP) Specialist**: Develops systems that can understand and generate human language, with applications in chatbots, language tra
nslation, and text analysis.
4. **Computer Vision Engineer**: Creates algorithms and systems that enable computers to interpret and understand visual data from images and videos, with applica
tions in self-driving cars, medical imaging, and security systems.
5. **AI Research Scientist**: Conducts research and development in AI, exploring new techniques and approaches to improve AI systems.
6. **Robotics Engineer**: Designs and develops intelligent systems that can interact with the physical world, with applications in manufacturing, healthcare, and
transportation.
```

Question 4: Create a 3-month roadmap for learning Generative AI from scratch.

```
You: Create a 3-month roadmap for learning Generative AI from scratch.

Career Assistant: Learning Generative AI from scratch requires a combination of theoretical knowledge, practical skills, and experience. Here's a 3-month roadmap
to help you get started:

**Month 1: Foundations and Fundamentals (Weeks 1-4)**

1. **Week 1-2: Introduction to Generative AI**
        * Learn the basics of Generative AI, including the different types (e.g., Markov models, Variational Autoencoders, Generative Adversarial Networks (GANs))
.
        * Understand the concepts of training data, model parameters, and loss functions.
        * Study popular Generative AI libraries and frameworks, such as TensorFlow, PyTorch, or Keras.
        * Recommended resources: "Generative Models" by Michael Welling, "Generative Adversarial Networks" by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
2. **Week 3-4: Math and Programming Fundamentals**
        * Review linear algebra, calculus, and probability.
        * Learn programming languages such as Python, MATLAB, or R.
        * Study numerical optimization techniques, such as gradient descent, and basic linear algebra.
        * Recommended resources: "Linear Algebra and Its Applications" by Gilbert Strang, "Python for Data Analysis" by Wes McKinney.
3. **Week 5: Generative Models and Loss Functions**
        * Study Markov models, Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs).
        * Learn how to implement these models using popular libraries and frameworks.
        * Understand the concept of training data, model parameters, and loss functions.
        * Recommended resources: "Markov Models" by John G. MacGregor, "Variational Autoencoders" by Joseph H. Kim, "Generative Adversarial Networks" by Ian Goodf
ellow, Yoshua Bengio, and Aaron Courville.
```

# Question 5: How can I improve my communication skills for a professional workplace?

